WAF PROJECT · GATEWAY INTEGRATION

Worklet 1 Integration Guide

v1.0 Adaptation manual for aligning Worklet 1 with the gateway-orchestrated architecture. Covers token JSON serialization, the in-memory API contract with the gateway, and the minimal changes needed to make W1 a clean stateless library.

PREPARED FOR	ISSUED BY	REVIEW DATE
Worklet 1 Team (Java)	W3 / Gateway Team	October 2026

Table of Contents

1. Executive Summary **2**

2. W1 Current Architecture - What is Already Correct **3**

3. The Token Flow Problem **4**

4. Change 1 - Add Token JSON Serializer **5**

5. Change 2 - Verify No W2/W3 References **8**

6. Integration Contract - W1 to Gateway API **10**

7. JSON Format Specification **12**

8. Migration Steps and Timeline **14**

9. What to Keep and What to Delete **15**

10. Testing and Acceptance Criteria **17**

1. Executive Summary

Worklet 1 (W1) is the foundational layer of the WAF pipeline - a stateless DFA-driven HTTP request inspector that scans for attack signatures and tokenizes JSON bodies. Unlike Worklet 2 (which requires substantial refactoring), **W1 is already 95% compatible** with the gateway-orchestrated architecture. The W1 team built it correctly from the start: as a pure library with immutable data structures, no HTTP server of its own, and no knowledge of downstream worklets.

This guide defines the minimal changes W1 needs to integrate cleanly with the new gateway architecture. The single most important addition is a **JSON serialization method** for the token stream - this is the bridge that allows the Java gateway to pass W1's output to the C++ W2 subprocess via stdin. Without this method, the gateway would have to duplicate W1's token structure knowledge.

Beyond the JSON serializer, W1 only needs to verify it has no lingering references to W2 or W3, and confirm that its public API matches what the gateway expects. The DFA engine, regex parser, tokenizer, and signature scanner all remain untouched.

The Two Required Changes

#	Change	Effort	Risk
1	Add tokensToJson() method to Worklet1Result	Low (2 hours)	Low
2	Verify no W2/W3 references exist in W1 code	Low (30 min)	None

Estimated total effort: 1 day. W1's architecture is sound. The DFA engine, Thompson's construction, subset construction, Hopcroft minimization, JSON tokenizer, and signature scanner all remain exactly as they are. Only the output boundary gets a new serialization method.

2. W1 Current Architecture - What is Already Correct

Before discussing changes, it is important to recognize what W1 has done right. The W1 team designed their component as a stateless library from day one, which is exactly what the gateway-orchestrated pattern requires. The table below maps W1's design decisions to the architectural properties they enable.

W1 Design Decision	Architectural Property Enabled
Worklet1 is a plain Java class (not a server)	Gateway can call it in-memory - no HTTP overhead
inspect(HttpRequest) returns Worklet1Result	Clean function-call API - gateway just invokes and uses the result
HttpRequest is an immutable Java record	Thread-safe: 1000 threads can call inspect() simultaneously with zero locks
Worklet1Result is an immutable Java record	Thread-safe: result cannot be corrupted by concurrent readers
Token is an immutable Java record	Thread-safe: tokens can be passed between threads without synchronization
All DFA tables are final (compiled once at startup)	Read-only after construction - no race conditions on shared state
No static mutable fields	No hidden global state that could cause thread interference
No HTTP server inside W1	No port conflicts, no network overhead for W1 calls
No calls to W2 or W3	W1 is decoupled - gateway orchestrates all cross-worklet communication
Worklet1Config loaded once at startup	Configuration is immutable after initialization - thread-safe by design

The key insight: W1's use of Java records (immutable data classes) is what makes the entire gateway architecture viable. Because W1 has no shared mutable state, the gateway can call `w1.inspect(request)` from 1000 concurrent threads without any locking, blocking, or race conditions. This is the gold standard for stateless library design.

3. The Token Flow Problem

The central question this guide answers is: **"If W1 does not call W2, how do W1's tokens reach W2?"** The answer is that W1 never talks to W2 directly. W1 returns Java Token objects to the gateway (an in-memory function call), and the gateway serializes those tokens to JSON text and pipes them to the W2 subprocess via stdin.

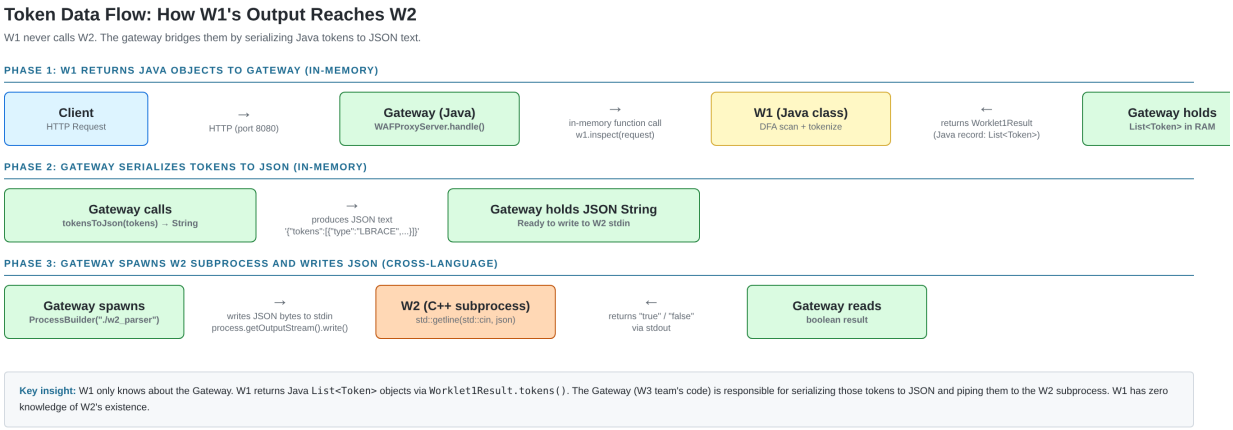


Figure 1. Token data flow - W1 returns Java objects to the gateway, which serializes them to JSON for W2.

The Three Forms of a Token

A single token exists in three different forms during its lifecycle, depending on which component is holding it:

Stage	Form	Location	Language
1. Created by W1	Java Token record (type, value, position)	JVM heap (RAM)	Java
2. Serialized by gateway	JSON text string	JVM heap (RAM), then stdin pipe	Text
3. Deserialized by W2	C++ Token struct (type, lexeme, line, column)	C++ process heap (RAM)	C++

There are no object files. Tokens are never written to disk. The entire flow happens in RAM. The JSON serialization is only used to cross the Java-to-C++ language boundary via the stdin pipe. Within Java, tokens remain as efficient Java records. Within C++, they become C++ structs. The JSON text is a transient serialization format that exists only for the few milliseconds it takes to cross the pipe.

4. Change 1 - Add Token JSON Serializer

W1 currently returns `List` from `Worklet1Result.tokens()`. This is perfect for in-memory Java use, but the gateway needs to send these tokens to the W2 C++ subprocess as JSON text via stdin. W1 must add a method that serializes its token list to the JSON format W2 expects.

The serializer can live in one of two places - W1 can choose whichever fits their codebase better:

Option A: Add a `tokensToJson()` method directly on the `Worklet1Result` record. Simplest, but adds serialization logic to a data class.

Option B: Create a separate `TokenJsonSerializer` utility class with a static `toJson(List)` method. Cleaner separation of concerns, follows single-responsibility principle.

Reference Implementation (Option B - recommended)

Part 1: Class structure and toJson() method

```
package com.wafgateway.worklet1.tokenizer;

import com.wafgateway.common.Token;
import com.wafgateway.common.TokenType;
import java.util.List;

/**
 * Serializes W1's token stream to JSON text for the W2 C++ subprocess.
 * Thread-safety: stateless, thread-safe. Can be called from
 * any number of threads concurrently.
 */
public final class TokenJsonSerializer {

    private TokenJsonSerializer() {
        // Utility class - no instances
    }

    /**
     * Serialize a list of tokens to a single-line JSON string.
     *
     * @param tokens  the token list from Worklet1Result.tokens()
     * @return JSON string ready to write to W2's stdin
     */
    public static String toJson(List<Token> tokens) {
        if (tokens == null || tokens.isEmpty()) {
            return "{\"tokens\":[]}";
        }

        StringBuilder sb = new StringBuilder();
        sb.append("{\"tokens\":[");

        for (int i = 0; i < tokens.size(); i++) {
            Token t = tokens.get(i);
            if (i > 0) sb.append(",");

            sb.append("{");
            sb.append("\"type\":\"").append(t.type().name()).append("\"");
            sb.append(",");
            sb.append("\"value\":\"").append(escapeJson(t.value())).append("\"");
            sb.append(",");
            sb.append("\"position\":").append(i);
            sb.append("}");
        }

        sb.append("]}");
        return sb.toString();
    }
    // escapeJson() continued on next page...
}
```

Part 2: JSON escaping helper method

```
    // Inside TokenJsonSerializer class (continued)

    /**
     * Escape special characters for JSON string values.
     * Handles backslash, double-quote, and control characters.
     */
    private static String escapeJson(String s) {
        if (s == null) return "";
        StringBuilder out = new StringBuilder(s.length() + 8);
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            switch (c) {
                case '\\': out.append("\\\\"); break;
                case '"':  out.append("\\\""); break;
                case '\n': out.append("\\n"); break;
                case '\r': out.append("\\r"); break;
                case '\t': out.append("\\t"); break;
                default:
                    if (c < 0x20) {
                        out.append(String.format("\\u%04x", (int) c));
                    } else {
                        out.append(c);
                    }
            }
        }
        return out.toString();
    }
}
```

Why W1 owns this serializer (not the gateway)

It might seem like the gateway could serialize the tokens itself. But that would require the gateway to know W1's Token structure (field names, types, ordering). If W1 ever changes its Token record (adds a field, renames a field), the gateway serializer would silently break. By keeping the serializer in W1, the W1 team owns the contract: they define the Token, and they define how it serializes. This is the single-responsibility principle in action.

5. Change 2 - Verify No W2/W3 References

W1 must be a pure library that knows nothing about W2 or W3. If W1 contains any code that calls W2, calls W3, or even references their class names, it breaks the gateway-orchestrated architecture. The gateway must be the only component that orchestrates cross-worklet communication.

Verification Procedure

Run the following grep commands from the project root. All of them should return **zero hits**. If any return results, those references must be removed.

```
# From the WAF-Firewall project root:

# Check for any references to Worklet 2
grep -ri "worklet2\|Worklet2\|w2\|W2" \
    src/main/java/com/wafgateway/worklet1/
# Expected: (no output)

# Check for any references to Worklet 3
grep -ri "worklet3\|Worklet3\|w3\|W3" \
    src/main/java/com/wafgateway/worklet1/
# Expected: (no output)

# Check for any HTTP client code (W1 should not make HTTP calls)
grep -ri "HttpClient\|HttpURLConnection\|RestTemplate" \
    src/main/java/com/wafgateway/worklet1/
# Expected: (no output)

# Check for any callback/delegate patterns
grep -ri "Callback\|Delegate\|Listener\|setWorklet" \
    src/main/java/com/wafgateway/worklet1/
# Expected: (no output)

# Check for any subprocess spawning (W1 should not spawn processes)
grep -ri "ProcessBuilder\|Runtime.exec\|Runtime.getRuntime" \
    src/main/java/com/wafgateway/worklet1/
# Expected: (no output)
```

What to do if you find references

If any of the grep commands above return results, W1 has coupling that must be removed. The table below lists common patterns and how to handle them:

Pattern Found	Action	Reason
Worklet2Dispatcher class	DELETE the class entirely	Gateway handles W2 invocation - W1 must not know W2 exists
setWorklet2Callback() method	DELETE the method and all callers	Callback pattern is for chained pipelines - gateway replaces it
HttpClient calls to W2 URL	DELETE the HTTP client code	W1 must not make network calls - only the gateway does

Import of com.wafgateway.worklet2.*	DELETE the import and dependent code	Cross-worklet imports violate decoupling
Import of com.wafgateway.worklet3.*	DELETE the import and dependent code	W1 must not depend on W3
ProcessBuilder spawning W2	DELETE the subprocess code	Gateway owns subprocess spawning, not W1

6. Integration Contract - W1 to Gateway API

The integration contract between W1 and the gateway is an in-memory Java function call. The gateway instantiates W1 once at startup, then calls `inspect()` for every incoming request. There is no HTTP, no serialization, no network - just a plain Java method invocation.

W1 Public API (what the gateway calls)

```
package com.wafgateway.worklet1;

import com.wafgateway.common.HttpRequest;
import com.wafgateway.common.Token;

public class Worklet1 {

    /**
     * Constructor - called ONCE at gateway startup.
     * Compiles all DFA rules, loads signatures from rules.txt.
     *
     * @param config  loaded from worklet1.properties
     */
    public Worklet1(Worklet1Config config) { ... }

    /**
     * Inspect an HTTP request for attack signatures and tokenize JSON body.
     *
     * THREAD-SAFE: Can be called from any number of threads concurrently.
     * The Worklet1 instance is shared across all request threads.
     *
     * @param request  immutable HttpRequest record
     * @return Worklet1Result with blocked flag, tokens, and session ID
     */
    public Worklet1Result inspect(HttpRequest request) { ... }
}

public record Worklet1Result(
    boolean blocked,           // true if attack detected
    BlockReason reason,        // why it was blocked
    List<Token> tokens,        // token stream (null if blocked or not JSON)
    String sessionId,          // extracted from Authorization header
    String detail              // human-readable description
) {
    /**
     * NEW METHOD: Serialize tokens to JSON for W2 subprocess.
     * Returns null if tokens is null or empty.
     */
    public String tokensToJson() {
        return TokenJsonSerializer.toJson(tokens);
    }
}
```

Gateway-side usage (what the gateway does)

```
// In WAFProxyServer - called once at startup
Worklet1Config config = Worklet1Config.load();
Worklet1 wafEngine = new Worklet1(config);  // Shared instance

// In the request handler - called per request, per thread
public void handleRequest(HttpExchange exchange) {
    // 1. Parse HTTP request into immutable record
    HttpRequest request = parseRequest(exchange);

    // 2. Call W1 (in-memory, thread-safe)
    Worklet1Result w1Result = wafEngine.inspect(request);

    // 3. If W1 blocked the request, drop it immediately
    if (w1Result.blocked()) {
        sendResponse(403, "Blocked: " + w1Result.detail());
        return;
    }

    // 4. Serialize tokens to JSON for W2
    String tokensJson = w1Result.tokensToJson();

    // 5. Spawn W2 subprocess and write JSON to stdin
    boolean w2Valid = w2Client.validateTokens(tokensJson);
    if (!w2Valid) {
        sendResponse(400, "Invalid JSON structure");
        return;
    }

    // 6. Call W3 (in-memory)
    String sessionId = w1Result.sessionId();
    String action = ActionExtractor.extract(method, path);
    boolean w3Valid = w3.checkValid(sessionId, action);
    // ... continue to backend ...
}
```

7. JSON Format Specification

The JSON format produced by `TokenJsonSerializer.toJson()` must match exactly what the W2 C++ subprocess expects to read from stdin. Any mismatch in field names, types, or structure will cause W2 to reject the input (return "false") or crash.

Token JSON Schema

```
{
  "tokens": [
    {
      "type": "LBRACE",       // TokenType enum name (uppercase)
      "value": "{",           // Raw lexeme from source text
      "position": 0           // Zero-based index in the token list
    },
    {
      "type": "KEY",
      "value": "action",
      "position": 1
    },
    {
      "type": "COLON",
      "value": ":",
      "position": 2
    },
    {
      "type": "STRING",
      "value": "CART",
      "position": 3
    },
    {
      "type": "RBRACE",
      "value": "}",
      "position": 4
    }
  ]
}
```

Field Specifications

Field	Type	Required	Description
tokens	array	Yes	Top-level array containing all tokens. Empty array if no body.
tokens[].type	string	Yes	TokenType enum name: LBRACE, RBRACE, LBRACKET, RBRACKET, COLON, COMMA, STRING, NUMBER, TRUE_TOK, FALSE_TOK, NULL_TOK, UNKNOWN.
tokens[].value	string	Yes	Raw lexeme from the source text. Must be JSON-escaped (backslash, quote, control chars).
tokens[].position	integer	Yes	Zero-based index in the token list. NOT the byte offset in the source - just the sequential position.

Critical formatting rules

- **Single-line output.** The JSON must be a single line with no newlines or pretty-printing. W2 uses `std::getline()` which reads until the first newline.

- **No trailing newline.** The gateway will add the newline when writing to stdin if needed. W1's serializer should not append one.

- **UTF-8 encoding.** All string values must be valid UTF-8. The gateway writes bytes using `getBytes("UTF-8")`.

- **Empty token list.** If the request has no body (e.g. GET request), output `{"tokens":[]}` - not null, not an empty string.

- **Consistent field ordering.** Always type, then value, then position. W2's JSON parser may be order-sensitive depending on implementation.

8. Migration Steps and Timeline

W1's migration is straightforward - only 2 phases, total effort approximately 1 day. Unlike W2 (which requires architectural restructuring), W1 only needs to add a serialization method and verify it has no coupling to other worklets.

Phase	Tasks	Deliverable	Effort
1. Add Token JSON Serializer	Create TokenJsonSerializer.java (or add tokensToJson() to Worklet1Result). Implement toJson() with proper JSON escaping. Write unit tests for the serializer (valid tokens, empty list, special characters). Verify output matches the JSON format specification in Section 7.	TokenJsonSerializer class compiles and passes unit tests. Output format matches W2 expectations exactly.	4 hours
2. Verify No W2/W3 Coupling	Run grep commands from Section 5. Remove any W2/W3 references found. Delete any Worklet2Dispatcher, callback, or HTTP client classes in W1. Verify W1 compiles cleanly after removals. Re-run existing W1 unit tests to confirm no regressions.	All grep commands return zero hits. W1 compiles with no dependencies on worklet2 or worklet3 packages. All existing W1 tests still pass.	2 hours

Phase 1 is the substantive work - writing the serializer and testing it. **Phase 2** is verification only - if W1 was built correctly from the start (which the code review suggests), this phase will find nothing to remove and take 30 minutes.

9. What to Keep and What to Delete

The single most important message of this guide: **W1's DFA engine is excellent and must remain unchanged.** The Thompson construction, subset construction, Hopcroft minimization, JSON tokenizer, and signature scanner are all correctly implemented and thoroughly tested. Only the output boundary gets a new serialization method.

Component	Action	Rationale
Worklet1 class (inspect method)	KEEP	Core API - gateway calls this directly
Worklet1Config	KEEP	Configuration loader - works perfectly
Worklet1Result record	KEEP	Immutable result - thread-safe by design
HttpRequest record	KEEP	Input contract - immutable, thread-safe
Token record	KEEP	Data structure - immutable, thread-safe
TokenType enum	KEEP	Token classification - used by serializer
AutomataCompiler (regex to DFA)	KEEP	Mathematical core - Thompson + Subset + Hopcroft
Nfa, Dfa classes	KEEP	Automaton data structures - runtime scanning
ThompsonBuilder	KEEP	Regex AST to NFA converter
SubsetConstruction	KEEP	NFA to DFA converter
HopcroftMinimizer	KEEP	DFA state minimizer
RegexParser, RegexNode, ByteSet	KEEP	Regex parsing infrastructure
SignatureScanner, RuleLoader, Rule	KEEP	Attack signature detection
JsonTokenizer, BodyTokenizer	KEEP	JSON lexical analysis - produces tokens
SessionIdValidator	KEEP	Session ID format validation
FieldExtractor, Field	KEEP	HTTP field extraction for scanning
rules.txt, worklet1.properties	KEEP	Configuration files - no changes
TokenJsonSerializer.java	ADD	NEW - serializes tokens to JSON for W2 subprocess
Any Worklet2Dispatcher class	DELETE	W1 must not know W2 exists
Any W2/W3 callback methods	DELETE	Gateway handles orchestration
Any HTTP client code in W1	DELETE	W1 is a library, not a client

Rule of thumb: If a file touches DFA, regex, tokenization, or signature scanning, keep it. If it touches W2, W3, HTTP clients, or callback dispatch, delete it. The DFA engine is W1's value - the only addition is the JSON serializer at the output boundary.

10. Testing and Acceptance Criteria

Before the W1 team declares the integration complete, the following acceptance tests must pass. These tests verify that the new JSON serializer produces correct output and that W1 has no lingering coupling to other worklets.

Serializer Tests (W1-1 through W1-6)

Test ID	Type	Test -> Expected Result
W1-1	Functional	Valid token list [LBRACE, KEY, COLON, STRING, RBRACE] -> valid JSON with 5 token objects
W1-2	Functional	Empty token list -> {"tokens":[]} (not null, not empty string)
W1-3	Edge case	Token with special character value (e.g. value contains quote) -> properly JSON-escaped output
W1-4	Edge case	Token with backslash in value -> backslash doubled in JSON output
W1-5	Format	Output is single-line JSON (no newlines, no pretty-printing)
W1-6	Format	Output has no trailing newline character

Coupling Verification Tests (W1-7 through W1-10)

Test ID	Type	Test -> Expected Result
W1-7	Coupling	grep for "worklet2" in W1 source -> zero hits
W1-8	Coupling	grep for "worklet3" in W1 source -> zero hits
W1-9	Coupling	grep for "HttpClient" in W1 source -> zero hits
W1-10	Coupling	grep for "ProcessBuilder" in W1 source -> zero hits

Integration Tests (W1-11 through W1-13)

Test ID	Type	Test -> Expected Result
W1-11	Integration	Gateway calls w1.inspect(request), receives Worklet1Result, calls tokensToJson(), writes to W2 stdin -> W2 returns "true" for valid JSON
W1-12	Integration	Gateway calls w1.inspect(request) on SQLi payload -> w1Result.blocked() is true, gateway returns 403 without calling W2
W1-13	Concurrency	100 threads call w1.inspect() simultaneously with different requests -> all succeed, no race conditions, no corrupted results

Verification Procedure

The W1 team can verify the serializer with a simple JUnit test:

```
@Test
void testTokensToJsonProducesValidOutput() {
    List<Token> tokens = List.of(
        new Token(TokenType.LBRACE, "{", 0, 0),
        new Token(TokenType.KEY, "action", 0, 1),
        new Token(TokenType.COLON, ":", 0, 8),
        new Token(TokenType.STRING, "CART", 0, 9),
        new Token(TokenType.RBRACE, "}", 0, 15)
    );

    String json = TokenJsonSerializer.toJson(tokens);

    // Verify it is valid JSON
    assertDoesNotThrow(() -> new JSONObject(json));

    // Verify structure
    JSONObject obj = new JSONObject(json);
    JSONArray arr = obj.getJSONArray("tokens");
    assertEquals(5, arr.length());
    assertEquals("LBRACE", arr.getJSONObject(0).getString("type"));
    assertEquals("{", arr.getJSONObject(0).getString("value"));
    assertEquals(0, arr.getJSONObject(0).getInt("position"));
}

@Test
void testEmptyTokenListProducesEmptyArray() {
    String json = TokenJsonSerializer.toJson(List.of());
    assertEquals("{\"tokens\":[]}", json);
}

@Test
void testNullTokenListProducesEmptyArray() {
    String json = TokenJsonSerializer.toJson(null);
    assertEquals("{\"tokens\":[]}", json);
}
```

Definition of Done

- All 13 acceptance tests pass (6 serializer + 4 coupling + 3 integration).
- TokenJsonSerializer produces output that W2 accepts (W2 returns "true" for valid token streams).
- No W2 or W3 references exist anywhere in the W1 source tree.
- W1 compiles cleanly with no dependencies on worklet2 or worklet3 packages.
- 100-thread concurrency test passes with no race conditions.
- All pre-existing W1 unit tests still pass (no regressions).

End of Guide. For questions about the JSON format or the gateway integration, contact the W3 / gateway team. For questions about the DFA engine or tokenizer, the existing W1 team owns those components and they remain unchanged.